

ГУАП

КАФЕДРА № 14

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ
ПРЕПОДАВАТЕЛЬ

12

ассистент

должность, уч. степень, звание

29.04.2016

подпись, дата

А.Ю Петров

инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ

Лабораторная работа №2. Перегрузка операторов

по курсу: Технология программирования

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ гр. №

1442

подпись, дата

И.Р Загидуллин
инициалы, фамилия

Санкт-Петербург 2026

Постановка задачи

Вариант 5:

Выполнить два задания

Создать класс «Дата», который содержит число, месяц и год. Перегрузить операторы: ++ (префиксная форма, метод) увеличивает на выбор пользователя либо дату, либо месяц, либо год на 1. -- (префиксная форма, метод) аналогично уменьшает. ++(постфиксная форма, дружественная функция) увеличивает на выбор пользователя либо дату, либо месяц, либо год на пользовательское число. -- (постфиксная форма, метод) аналогично уменьшает. • Задание 2. Бинарная операция.

Создать объект типа очередь. Перегрузить оператор + как функцию член и * как дружественную функцию. + добавляет элемент в очередь, * умножает элементы в очереди на пользовательское число. Вытаскивает элемент из очереди оператор -. Очереди можно присваивать (=), проверять на равенство (поэлементное) == или !=, вводить и выводить в поток.

Необходимо выполнить разделение на h и srr файлы для каждого класса. h файлы содержат определение, srr файлы содержат реализацию. функция main обязана располагаться в отдельном srr файле.

Элемент очереди/стека/списка содержит данные и ссылку на предыдущий/следующий/предыдущий и следующий элемент. Элемент реализовать с помощью класса или структуры. В классе или структуре, реализующей элемент, необходимо поместить функции для установки и извлечения данных. То есть напрямую обращения к данным и ссылке(-ам) быть не должно.

Необходимо работать с динамическим выделением памяти. Важно выделять и всегда очищать выделенную память. Необходимо написать для каждого класса: конструктор с параметрами и без параметров, конструктор копирования, деструктор, необходимо продемонстрировать применение спецификатора explicit.

Требуется реализовать методы/функции с аргументами по умолчанию. Реализовать пользовательское меню согласно заданию. В пользовательском меню обязательно предоставить пользователю возможность взаимодействовать со всеми настраиваемыми параметрами. То есть не должно существовать в программе числовых или буквенных констант, которые могут быть введены пользователем.

1. Формализация

Входные данные для задания 1: Программа принимает на вход начальную дату в формате “день месяц год”. День, месяц и год — целые числа. Затем пользователь выбирает из меню команду: 1 — префиксный ++, 2 — префиксный --, 3 — постфиксный ++, 4 —

постфиксный --, 0 — выход. Для команд 1 и 2 дополнительно вводится выбор изменяемого компонента (1-день, 2-месяц, 3-год). Для команд 3 и 4 дополнительно вводится выбор изменяемого компонента и значение, на которое изменить дату (целое число).

Выходные данные для задания 1: Программа выводит текущую дату после выполнения каждой команды в формате “день.месяц.год”. Для постфиксных операций дополнительно выводится старая дата до изменения. В случае ошибки выбора команды или компонента программа выводит сообщение “wrong answer”.

Входные данные для задания 2: Программа не принимает начальную очередь. Пользователь выбирает из меню команду: 1 — добавить элемент (+), 2 — умножить элементы (*), 3 — вытащить элемент (-), 4 — присвоить очередь (=), 5 — сравнить очереди (== и !=), 6 — конструктор копирования, 0 — выход. Для команды 1 дополнительно вводится целое число — добавляемый элемент. Для команды 2 дополнительно вводится целое число — множитель. Для команды 4 дополнительно вводится количество элементов и сами элементы для создания исходной очереди. Для команды 5 дополнительно вводится количество элементов и сами элементы для создания второй очереди.

Выходные данные для задания 2: Программа выводит текущее состояние очереди после каждой операции в формате [a, b, c]. Для команды 3 дополнительно выводится извлечённое значение. Для команды 5 выводится результат сравнения (“Queues are EQUAL” или “Queues are NOT EQUAL”). При попытке извлечь элемент из пустой очереди выводится сообщение “Queue is empty”. В случае ошибки выбора команды выводится сообщение “Invalid choice!”.

2. Исходный код

```
#pragma once

struct Node
{
    int data;
    Node* next;

    Node(int value)
    {
        data = value;
        next = nullptr;
    }
};
```

Листинг 1 - Файл Node.h

```
#pragma once
#include <iostream>

class Date
```

```

{
protected:
    int day;
    int month;
    int year;

public:
    Date(int current_day, int current_month, int current_year);

    int askChoice();
    void showMenu() const;

    // Количество дней в месяце (с учётом високосного года)
    int daysInMonth(int m, int y) const;

    // Нормализация даты
    void normalize();

    // префиксный ++
    Date& operator++();

    // префиксный --
    Date& operator--();

    void display() const;

    // постфиксный ++
    friend Date operator++(Date& acc, int);

    // постфиксный --
    Date operator--(int);
};

```

Листинг 2 - Файл Date.h

```

#include <iostream>
#include "Date.h"

// отсюда

Date::Date(int current_day, int current_month, int current_year)
{
    day = current_day;
    month = current_month;
    year = current_year;
}

int Date::askChoice()
{
    int choice;
    std::cout << "change? (1-day, 2-month, 3-year): ";
    std::cin >> choice;
    return choice;
}

int Date::daysInMonth(int m, int y) const
{
    if (m == 2) {
        // Високосный год
        if ((y % 4 == 0 && y % 100 != 0) || (y % 400 == 0))
            return 29;
        return 28;
    }
    if (m == 4 || m == 6 || m == 9 || m == 11) return 30;
    return 31;
}

```

```

void Date::normalize()
{
    // Корректируем месяцы
    while (month > 12) {
        month -= 12;
        year++;
    }
    while (month < 1) {
        month += 12;
        year--;
    }

    // Корректируем дни
    while (day > daysInMonth(month, year)) {
        day -= daysInMonth(month, year);
        month++;
        if (month > 12) {
            month -= 12;
            year++;
        }
    }
    while (day < 1) {
        month--;
        if (month < 1) {
            month += 12;
            year--;
        }
        day += daysInMonth(month, year);
    }
}

void Date::display() const
{
    std::cout << day << "." << month << "." << year;
}

// префиксный ++
Date& Date::operator++()
{
    int choice = askChoice();
    switch (choice)
    {
        case 1:
            day++;
            break;
        case 2:
            month++;
            break;
        case 3:
            year++;
            break;
        default:
            std::cout << "wrong answer";
            break;
    }
    normalize();
    return *this;
}

// префиксный --
Date& Date::operator--()
{
    int choice = askChoice();
    switch (choice)
    {

```

```

        case 1:
            day--;
            break;
        case 2:
            month--;
            break;
        case 3:
            year--;
            break;
        default:
            std::cout << "wrong answer";
            break;
    }
    normalize();
    return *this;
}

// постфиксный ++
Date operator++(Date& acc, int)
{
    Date temp = acc;
    int choice = acc.askChoice();
    int value;
    std::cout << "how much to increase?: ";
    std::cin >> value;

    switch (choice)
    {
        case 1:
            acc.day += value;
            break;
        case 2:
            acc.month += value;
            break;
        case 3:
            acc.year += value;
            break;
        default:
            std::cout << "wrong answer";
            break;
    }
    acc.normalize();
    return temp;
}

// постфиксный --
Date Date::operator--(int)
{
    Date temp = *this;
    int choice = askChoice();
    int value;
    std::cout << "how much to decrease?: ";
    std::cin >> value;

    switch (choice)
    {
        case 1:
            day -= value;
            break;
        case 2:
            month -= value;
            break;
        case 3:
            year -= value;
            break;
    }
}

```

```

        default:
            std::cout << "wrong answer";
            break;
    }
    normalize();
    return temp;
}

void Date::showMenu() const
{
    std::cout << "\n=== Current date: ";
    display();
    std::cout << " ===" << std::endl;
    std::cout << "1. Prefix ++ (increase by 1)" << std::endl;
    std::cout << "2. Prefix -- (decrease by 1)" << std::endl;
    std::cout << "3. Postfix ++ (increase by user value)" << std::endl;
    std::cout << "4. Postfix -- (decrease by user value)" << std::endl;
    std::cout << "0. Exit" << std::endl;
    std::cout << "Your choice: ";
}

```

Листинг 3 - Файл Date.cpp

```

#pragma once
#include <iostream>

struct Node
{
    int data;
    Node* next;

    Node(int value)
    {
        data = value;
        next = nullptr;
    }
};

class Queue
{
private:
    Node* head;
    Node* tail;

public:
    Queue();
    Queue(const Queue& other);
    ~Queue();

    Queue& operator+(int value);
    friend Queue operator*(const Queue& q, int factor);
    int operator-();
    Queue& operator=(const Queue& other);
    bool operator==(const Queue& other) const;
    bool operator!=(const Queue& other) const;

    friend std::ostream& operator<<(std::ostream& os, const Queue& q);
    friend std::istream& operator>>(std::istream& is, Queue& q);

    void display() const;
};

```

Листинг 4 - Файл Queue.h

```

#include "Queue.h"

Queue::Queue()
{
    head = nullptr;
    tail = nullptr;
}

Queue::Queue(const Queue& other)
{
    head = nullptr;
    tail = nullptr;

    Node* current = other.head;
    while (current != nullptr)
    {
        *this + current->data;
        current = current->next;
    }
}

Queue::~Queue()
{
    while (head != nullptr)
    {
        Node* temp = head;
        head = head->next;
        delete temp;
    }
    tail = nullptr;
}

Queue& Queue::operator+(int value)
{
    Node* newNode = new Node(value);

    if (head == nullptr)
    {
        head = newNode;
        tail = newNode;
    }
    else
    {
        tail->next = newNode;
        tail = newNode;
    }

    return *this;
}

Queue operator*(const Queue& q, int factor)
{
    Queue result;
    Node* current = q.head;

    while (current != nullptr)
    {
        result + (current->data * factor);
        current = current->next;
    }

    return result;
}

int Queue::operator-()

```



```

{
    if (head == nullptr)
    {
        std::cout << "Queue is empty!" << std::endl;
        return -1;
    }

    Node* temp = head;
    int value = head->data;
    head = head->next;

    if (head == nullptr)
    {
        tail = nullptr;
    }

    delete temp;
    return value;
}

Queue& Queue::operator=(const Queue& other)
{
    if (this == &other)
    {
        return *this;
    }

    // Очищаем текущую очередь
    while (head != nullptr)
    {
        Node* temp = head;
        head = head->next;
        delete temp;
    }
    tail = nullptr;

    // Копируем из other
    Node* current = other.head;
    while (current != nullptr)
    {
        *this + current->data;
        current = current->next;
    }

    return *this;
}

bool Queue::operator==(const Queue& other) const
{
    Node* a = head;
    Node* b = other.head;

    while (a != nullptr && b != nullptr)
    {
        if (a->data != b->data)
        {
            return false;
        }
        a = a->next;
        b = b->next;
    }

    return a == nullptr && b == nullptr;
}

```

```

bool Queue::operator!=(const Queue& other) const
{
    return !(*this == other);
}

std::ostream& operator<<(std::ostream& os, const Queue& q)
{
    Node* current = q.head;
    os << "[";
    while (current != nullptr)
    {
        os << current->data;
        if (current->next != nullptr)
        {
            os << ", ";
        }
        current = current->next;
    }
    os << "]";
    return os;
}

std::istream& operator>>(std::istream& is, Queue& q)
{
    int value;
    std::cout << "Enter value to add: ";
    is >> value;
    q + value;
    return is;
}

void Queue::display() const
{
    if (head == nullptr)
    {
        std::cout << "[";
        return;
    }

    Node* current = head;
    std::cout << "[";
    while (current != nullptr)
    {
        std::cout << current->data;
        if (current->next != nullptr)
        {
            std::cout << ", ";
        }
        current = current->next;
    }
    std::cout << "]";
}

```

Листинг 5 - Файл Queue.cpp

```

#include <iostream>
#include <locale>
#include "Date.h"
#include "Queue.h"

void dateMenu()
{
    int day, month, year;

    std::cout << "\n===== DATE PROGRAM =====\n" << std::endl;
}

```

```

std::cout << "Enter initial date:" << std::endl;
std::cout << "Day: ";
std::cin >> day;
std::cout << "Month: ";
std::cin >> month;
std::cout << "Year: ";
std::cin >> year;

Date currentDate(day, month, year);

int choice;
do {
    currentDate.showMenu();
    std::cin >> choice;

    switch (choice) {
        case 1:
            ++currentDate;
            std::cout << "After prefix ++: ";
            currentDate.display();
            std::cout << std::endl;
            break;
        case 2:
            --currentDate;
            std::cout << "After prefix --: ";
            currentDate.display();
            std::cout << std::endl;
            break;
        case 3: {
            Date oldDate = currentDate++;
            std::cout << "After postfix ++: ";
            currentDate.display();
            std::cout << " (old value: ";
            oldDate.display();
            std::cout << ")" << std::endl;
            break;
        }
        case 4: {
            Date oldDate = currentDate--;
            std::cout << "After postfix --: ";
            currentDate.display();
            std::cout << " (old value: ";
            oldDate.display();
            std::cout << ")" << std::endl;
            break;
        }
        case 0:
            std::cout << "Exiting Date program..." << std::endl;
            break;
        default:
            std::cout << "Invalid choice!" << std::endl;
    }
} while (choice != 0);
}

void queueMenu()
{
    Queue q;
    int choice;

    do {
        std::cout << "\n===== QUEUE PROGRAM =====\n" << std::endl;
        std::cout << "Current queue: ";
        q.display();
        std::cout << std::endl;
    }
}

```

```

std::cout << "1. Add element (+)" << std::endl;
std::cout << "2. Multiply all elements by number (*)" << std::endl;
std::cout << "3. Extract element (-)" << std::endl;
std::cout << "4. Assign queue (=)" << std::endl;
std::cout << "5. Compare two queues (== and !=)" << std::endl;
std::cout << "6. Copy constructor" << std::endl;
std::cout << "0. Exit" << std::endl;
std::cout << "Your choice: ";
std::cin >> choice;

switch (choice) {
case 1: {
    int value;
    std::cout << "Enter value to add: ";
    std::cin >> value;
    q + value;
    std::cout << "Queue after adding: ";
    q.display();
    std::cout << std::endl;
    break;
}
case 2: {
    int factor;
    std::cout << "Enter multiplier: ";
    std::cin >> factor;
    q = q * factor;
    std::cout << "Queue after multiplication: ";
    q.display();
    std::cout << std::endl;
    break;
}
case 3: {
    Queue emptyQueue;
    if (q == emptyQueue)
    {
        std::cout << "Queue is empty!" << std::endl;
    }
    else
    {
        int value = -q;
        std::cout << "Extracted value: " << value << std::endl;
        std::cout << "Queue after extraction: ";
        q.display();
        std::cout << std::endl;
    }
    break;
}
case 4: {
    Queue q2;
    int n;
    std::cout << "How many elements to add to source queue? ";
    std::cin >> n;
    for (int i = 0; i < n; ++i) {
        int val;
        std::cout << "Enter value " << i + 1 << ": ";
        std::cin >> val;
        q2 + val;
    }
    std::cout << "Source queue: ";
    q2.display();
    std::cout << std::endl;
    q = q2;
    std::cout << "After assignment, current queue: ";
    q.display();
    std::cout << std::endl;
}
}

```

```

        break;
    }
    case 5: {
        Queue q2;
        int n;
        std::cout << "How many elements in second queue? ";
        std::cin >> n;
        for (int i = 0; i < n; ++i) {
            int val;
            std::cout << "Enter value " << i + 1 << ": ";
            std::cin >> val;
            q2 + val;
        }
        std::cout << "Current queue: ";
        q.display();
        std::cout << std::endl;
        std::cout << "Second queue: ";
        q2.display();
        std::cout << std::endl;

        if (q == q2) {
            std::cout << "Queues are EQUAL" << std::endl;
        }
        else {
            std::cout << "Queues are NOT EQUAL" << std::endl;
        }
        break;
    }
    case 6: {
        Queue q2 = q;
        std::cout << "Original queue: ";
        q.display();
        std::cout << std::endl;
        std::cout << "Copied queue: ";
        q2.display();
        std::cout << std::endl;

        q2 + 999;
        std::cout << "After adding 999 to copied queue:" << std::endl;
        std::cout << "Original: ";
        q.display();
        std::cout << std::endl;
        std::cout << "Copied: ";
        q2.display();
        std::cout << std::endl;
        break;
    }
    case 0:
        std::cout << "Exiting Queue program..." << std::endl;
        break;
    default:
        std::cout << "Invalid choice!" << std::endl;
    }
} while (choice != 0);
}

int main()
{
    setlocale(LC_ALL, "Russian");

    int labChoice;

    std::cout << "===== LABORATORY WORK =====" << std::endl;
    std::cout << "1. Task 1: Date (Unary operations)" << std::endl;
    std::cout << "2. Task 2: Queue (Binary operations)" << std::endl;

```

```

std::cout << "0. Exit" << std::endl;
std::cout << "Your choice: ";
std::cin >> labChoice;

switch (labChoice) {
case 1:
    dateMenu();
    break;
case 2:
    queueMenu();
    break;
case 0:
    std::cout << "Goodbye!" << std::endl;
    break;
default:
    std::cout << "Invalid choice!" << std::endl;
}

return 0;
}

```

Листинг 6 - Файл main.cpp

3. Результат работы программы

```

===== LABORATORY WORK =====
1. Task 1: Date (Unary operations)
2. Task 2: Queue (Binary operations)
0. Exit
Your choice: 1

===== DATE PROGRAM =====

Enter initial date:
Day: 15
Month: 06
Year: 2006

=== Current date: 15.6.2006 ===
1. Prefix ++ (increase by 1)
2. Prefix -- (decrease by 1)
3. Postfix ++ (increase by user value)
4. Postfix -- (decrease by user value)
0. Exit
Your choice: 1
change? (1-day, 2-month, 3-year): 2
After prefix ++: 15.7.2006

=== Current date: 15.7.2006 ===
1. Prefix ++ (increase by 1)
2. Prefix -- (decrease by 1)
3. Postfix ++ (increase by user value)
4. Postfix -- (decrease by user value)
0. Exit
Your choice: 4
change? (1-day, 2-month, 3-year): 3
how much to decrease?: 6
After postfix --: 15.7.2000 (old value: 15.7.2006)

=== Current date: 15.7.2000 ===
1. Prefix ++ (increase by 1)
2. Prefix -- (decrease by 1)
3. Postfix ++ (increase by user value)
4. Postfix -- (decrease by user value)
0. Exit
Your choice: 3
change? (1-day, 2-month, 3-year): 1
how much to increase?: 17
After postfix ++: 1.8.2000 (old value: 15.7.2000)

=== Current date: 1.8.2000 ===
1. Prefix ++ (increase by 1)
2. Prefix -- (decrease by 1)
3. Postfix ++ (increase by user value)
4. Postfix -- (decrease by user value)
0. Exit
Your choice: |

```

Листинг 7 - Работа программы задание 1.

```

Current queue: []
1. Add element (+)
2. Multiply all elements by number (*)
3. Extract element (-)
4. Assign queue (=)
5. Compare two queues (== and !=)
6. Copy constructor
0. Exit
Your choice: 1
Enter value to add: 5
Queue after adding: [5]

===== QUEUE PROGRAM =====

Current queue: [5]
1. Add element (+)
2. Multiply all elements by number (*)
3. Extract element (-)
4. Assign queue (=)
5. Compare two queues (== and !=)
6. Copy constructor
0. Exit
Your choice: 7
Invalid choice!

===== QUEUE PROGRAM =====

Current queue: [5]
1. Add element (+)
2. Multiply all elements by number (*)
3. Extract element (-)
4. Assign queue (=)
5. Compare two queues (== and !=)
6. Copy constructor
0. Exit
Your choice: 1
Enter value to add: 8
Queue after adding: [5, 8]

===== QUEUE PROGRAM =====

Current queue: [5, 8]
1. Add element (+)
2. Multiply all elements by number (*)
3. Extract element (-)
4. Assign queue (=)
5. Compare two queues (== and !=)
6. Copy constructor
0. Exit

```

Листинг 8 - Работа программы задание 2.

```

Current queue: [5, 8, 15]
1. Add element (+)
2. Multiply all elements by number (*)
3. Extract element (-)
4. Assign queue (=)
5. Compare two queues (== and !=)
6. Copy constructor
0. Exit
Your choice: 3
Extracted value: 5
Queue after extraction: [8, 15]

===== QUEUE PROGRAM =====

Current queue: [8, 15]
1. Add element (+)
2. Multiply all elements by number (*)
3. Extract element (-)
4. Assign queue (=)
5. Compare two queues (== and !=)
6. Copy constructor
0. Exit
Your choice: 6
Original queue: [8, 15]
Copied queue: [8, 15]
After adding 999 to copied queue:
Original: [8, 15]
Copied: [8, 15, 999]

===== QUEUE PROGRAM =====

Current queue: [8, 15]
1. Add element (+)
2. Multiply all elements by number (*)
3. Extract element (-)
4. Assign queue (=)
5. Compare two queues (== and !=)
6. Copy constructor
0. Exit
Your choice: 5
How many elements in second queue? 7
Enter value 1: 5 6 231 56 5 1 0
Enter value 2: Enter value 3: Enter value 4: Enter value 5: Enter value 6: Enter value 7: Current queue: [8, 15]
Second queue: [5, 6, 231, 56, 5, 1, 0]
Queues are NOT EQUAL

===== QUEUE PROGRAM =====

Current queue: [8, 15]
1. Add element (+)
2. Multiply all elements by number (*)
3. Extract element (-)
4. Assign queue (=)
5. Compare two queues (== and !=)

```

Листинг 9 - Работа программы задание 2.

4. Вывод

В ходе выполнения лабораторной работы были выполнены два задания:

- Задание 1 Унарная операция. Создан класс «Дата», который содержит день, месяц и год. Были перегружены операторы ++, --, префиксная и постфиксная форма, для методов и дружественных функций класса.
- Задание 2 Бинарная операция. Был создан объект типа очередь, с помощью перегрузки операторов, которых были написаны следующие функции: добавление элементов в очередь, путем перегрузки оператора ++, умножение элементов очереди на число, получение элемента из очереди, сравнение, ввод и вывод в поток.

Были поставленные цели были выполнены.